COEN 346
Operating Systems
Winter 2026

Programming Assignment 2 Report

Nirthika Ilaiyarajah (40298669)
Ruhani Kareer (40315722)

Presented to:
Professor Bahareh Goodarzi

Concordia University
Due date: 08/03/2026

1. Introduction

The Dining Philosophers problem is a classic synchronization challenge where philosophers share limited chopsticks to eat, risking deadlock and starvation. This assignment extends the problem by allowing philosophers to talk, with mutual exclusion, and adding two pepper shakers that must be shared among eating philosophers. Our solution implements a monitor using Java's wait() and notifyAll() with a hunger counter strategy to ensure deadlock-free and starvation-free operation.

2. Description of the methods

This program is composed of four classes that work together to solve the problem:

- **DiningPhilosophers.java:** contains the main() method and starts the program by creating the monitor and philosopher threads.
- **Philosopher.java:** Represents each philosopher as a thread and defines the actions of eating, thinking, and talking during the dining cycle.
- **BaseThread.java:** Is an extension of the Philosopher class and provides basic thread functionality and assigns a unique thread ID to each philosopher thread.
- **Monitor.java:** Manages synchronization between philosophers and controls access to shared resources such as chopsticks, pepper shakers, and talking permission.

2.1. DiningPhilosophers Class

Method	Description
`main(String[] argv)`	The main method is the starting point of the program. It reads the number of philosophers from user input, creates the shared monitor, and then creates and starts the philosopher threads. It also waits for all philosopher threads to finish their dining steps before terminating the program.
`getNumberOfPhilosophers()`	This method returns the total number of philosophers participating in the simulation. It allows other parts of the program, such as the monitor or philosopher threads, to access this value when they need to know how many philosophers exist.
`reportException(Exception e)`	This method is used for error reporting. When an exception occurs, it prints the exception type, message, and stack trace to the error output (System.err) to help identify and debug problems in the program.

2.2. Philosopher Class

Method	Description
`eat()`	The eat() method simulates eating by printing when the philosopher starts eating, let the other threads run yield, sleep for a random time (time to eat), let other threads run again and finally prints that eating is done.
`think()`	The think() method simulates the act of thinking by printing when the philosopher starts thinking, let the other threads run, sleep for a random time (time to think), let other threads run again and finally prints that thinking is done.
`talk()`	The talk() method simulates the act of talking by printing when a philosopher starts talking, let the other threads run, say something at random, let other threads run again and finally print that talking is done.
`run()`	The run() method defines the main behaviour of each philosopher thread. It repeatedly performs the dining cycle: requesting forks from the monitor, eating, releasing forks, thinking, and occasionally talking. This loop runs for a fixed number of dining steps, simulating the activities of philosophers sharing resources concurrently.
`saySomething()`	The saySomething() method prints a random phrase that represents the philosophers speaking. It stores several phrases in a string array, then uses Math.random() to select one phrase at random. The chosen phrase is printed along with the philosopher's TID.

2.3. Monitor Class

Method	Description
`putDown(int piTID)`	This method is called when a philosopher becomes hungry and wants to eat. The philosopher's state is set to HUNGRY and the monitor checks whether both neighboring philosophers are not eating. If they are eating, the philosopher waits using wait(). When both chopsticks become available, the philosopher is allowed to eat.
`putDown(int piTID)`	This method is called after a philosopher finish eating. The philosopher releases the chopsticks and changes their state to THINKING. The monitor then checks whether neighboring philosophers can now eat and notifies waiting threads using notifyAll().

`requestTalk()`	This method ensures that only one philosopher can talk at a time. If another philosopher is already talking, the calling philosopher waits using wait(). Once the monitor allows it, the philosopher can begin talking.
`endTalk()`	This method is called when a philosopher finishes talking. The monitor sets the talking flag to false and calls notifyAll() so that other waiting philosophers can talk.
`test(int philosopherId)`	This helper method checks whether a philosopher is allowed to start eating. It verifies that the philosopher is in the HUNGRY state and that neither the left nor the right neighbor is currently eating. It also compares hunger counters of neighboring philosophers to give priority to the one who has waited longer. If all conditions are satisfied, the philosopher's state is changed to EATING.
`requestPepperShakers()`	We ensure with this method that philosophers can only eat when two pepper shakers are available. If fewer than two pepper shakers are available, the philosopher waits.
`releasePepperShakers()`	This method releases the pepper shakers after the philosopher finishes eating and notifies waiting philosophers that the resource is available again.

3. Flow of the Program

1) The program starts execution in the main() method of DiningPhilosophers.java.

2) User is prompted to input the number of philosophers.

3) A Monitor object is created to control synchronization between philosophers.

4) The program creates an array of philosopher threads and starts them.

5) Each Philosopher performs in a loop these actions:

- Request permission to eat
- Obtain pepper shakers
- Eat
- releases chopsticks
- Think
- Optionally, talk if no other philosopher is talking

6) The main thread waits for all philosopher threads to finish using join() method.

7) The program terminates once all philosophers finish their dining steps.

4. **Synchronization Solution**

   In this program, the synchronization is managed by the Monitor class, which acts as the central controller and uses the following Java built-in methods: synchronized, wait(), notifyAll(). All the synchronized methods previously explained in the Monitor class ensure mutual exclusion, meaning that only one thread can execute a synchronized method at a time. The wait() method is used when a philosopher must wait for a resource and in a similar way, notifyAll() wakes up all waiting philosophers when the system state changes, so they can try to execute tasks such as eating, talking and take pepper.

   Philosophers need to contact the Monitor class before they can use shared resources such as state[], hungerCounter[], and availablePepperShakers []and they cannot be modified simultaneously. The Monitor first checks both neighbouring philosophers before granting eating permission to the philosopher who wants to eat. When the philosopher cannot eat, he must wait until he receives approval to use resources. Also, only one philosopher is allowed to talk at a time. In addition to these tasks, it also handles the use of pepper shakers by allowing them to take only when both pepper shakers are available.

   In conclusion, we used a monitor-based system to ensure safe resource sharing rather than semaphores because monitor provides a simpler approach, as Java already has the important wait() and notifyAll() methods.

5. **Starvation-free Method**

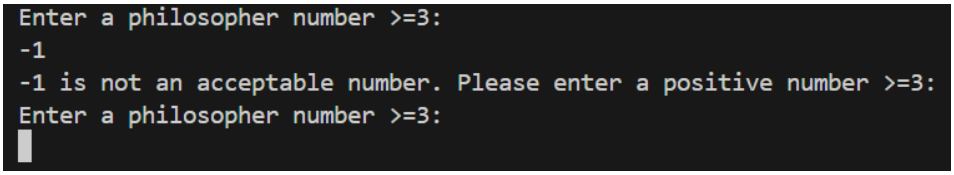
   The method in the Monitor class that ensures a starvation-free solution is test(). There, we use an array called hungerCount[], which increases the hunger count of the philosophers each time they become hungry. When the monitor checks whether a philosopher can start eating, it doesn't only check if the neighbours are also hungry, it also compares the hunger count, which indicates how long they have been waiting. If one neighbour is also hungry and has waited longer, the priority is given to him. This way ensures that no threads are in starvation.

6. **Deadlock-free Method**

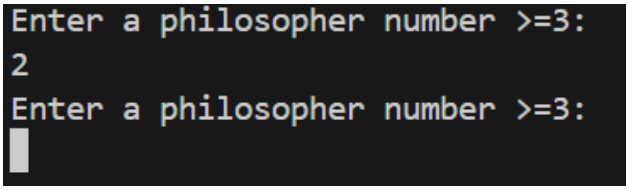
The pickUp() method helps prevent deadlocks because it uses the test() method found in the Monitor class. The test() method checks whether both neighbouring philosophers are not eating before allowing a philosopher to eat. If the philosopher cannot eat, the wait() method blocks the thread until the monitor updates the system state and notifies the waiting philosophers. The putDown() method uses notifyAll() to send a notification when a philosopher finishes eating. A philosopher can only eat when neither of his neighbours is eating. The system prevents deadlocks by using monitor-based resource access control and condition enforcement to stop circular waiting.

7. Results

Output when negative input is given:



Output when a number less than 3 is entered:



After entering a valid input value (3), the system initialized the monitor and created three philosopher threads. The following image shows part of the program execution:

The output demonstrates that the monitor correctly controls access to shared resources. A philosopher can only begin eating when both chopsticks are available. When a philosopher finishes eating, the chopsticks are released so that neighboring philosophers can proceed. The system continuously cycles through thinking, eating, and talking states without stopping, which indicates that no deadlock occurs.

The additional constraint regarding the pepper shakers is also enforced. The output shows messages such as *"Philosopher X got pepper shakers to eat"* and *"Philosopher X released*

pepper shakers", confirming that the pepper shakers are treated as shared resources and must be acquired before eating.

The output also confirms that only one philosopher is allowed to talk at a time. Other philosophers must wait until the current philosopher finishes speaking before another can begin talking.

Furthermore, the program ensures that philosophers do not starve. Each philosopher repeatedly transitions between thinking, talking, and eating states. The output shows that every philosopher eventually gets access to the shared resources and performs all actions multiple times during the simulation.

```
Enter a philosopher number >=3:
3
Monitor initialized with 3 philosophers.
3 philosopher(s) came in for a dinner.
Philosopher 1 is hungry (waited 1 times)
Philosopher 1 starts eating.
Philosopher 1 got pepper shakers to eat.
Philosopher 2 is hungry (waited 1 times)
Philosopher 3 is hungry (waited 1 times)
Philospher 1 has started eating.
Philospher 1 is done eating.
Philosopher 1 released pepper shakers.
Philosopher 1 finished eating, is now thinking.
Philosopher 2 starts eating.
Philosopher 2 got pepper shakers to eat.
Philospher 2 has started eating.
Philospher 1 has started thinking.
Philosopher 3 starts eating.
Philospher 1 is done thinking.
A philosopher started talking.
Philosopher 1 has started talking.
Philosopher 1 says: 2 + 2 = 5 for extremely large values of 2...
Philosopher 1 is done talking.
A philosopher finished talking.
```

Finally, the program terminates normally after the philosophers finish their execution:

```
All philosophers have left. System terminates normally.
```

8. Conclusion

In this assignment, the synchronization problem was solved using a monitor-based approach implemented with Java's built-in synchronization primitives. The system successfully coordinates multiple philosopher threads while preventing deadlock and starvation. Philosophers are only allowed to eat when both chopsticks are available, and the monitor ensures that access to shared resources is properly controlled. Additional constraints were also implemented successfully. Only one philosopher is allowed to talk at a time, and two pepper shakers are shared among eating philosophers using mutual exclusion. The program also supports a variable number of philosophers entered by the user and validates the input to ensure it meets the required conditions. Overall, the implementation demonstrates correct thread synchronization and proper management of shared resources in a concurrent system. The results show that the program runs continuously and terminates normally while satisfying all constraints of the problem.